

MiniTomcat Server



```
minitomcat/
├── pom.xml
├── src/main/java/com/example/minitomcat/
│   ├── server/
│   │   ├── MiniTomcat.java           // accept loop + lifecycle
│   │   ├── ServerConfig.java        // port, pool size, timeouts
│   │   └── ConnectionWorker.java     // Runnable: handles one socket
│   ├── http/
│   │   ├── HttpRequest.java
│   │   ├── HttpRequestParser.java
│   │   ├── HttpResponse.java
│   │   ├── HttpMethod.java
│   │   └── HttpStatus.java
│   ├── concurrent/
│   │   ├── ServerThreadPool.java     // wraps ThreadPoolExecutor
│   │   └── RequestContextHolder.java // ThreadLocal request context
│   ├── pool/
│   │   ├── ObjectPool.java          // generic pool interface
│   │   └── BufferPool.java           // pooled byte[]/ByteBuffer
│   ├── jdbc/
│   │   ├── MiniConnectionPool.java
│   │   └── PooledConnection.java
│   ├── servlet/
│   │   ├── MiniServlet.java
│   │   ├── MiniServletConfig.java
│   │   ├── MiniServletContext.java
│   │   ├── MiniHttpServletRequest.java
│   │   ├── MiniHttpServletResponse.java
│   │   ├── MiniFilter.java
│   │   └── MiniServletContainer.java // path -> servlet routing
│   ├── bridge/
│   │   └── MiniDispatcherServlet.java // MiniTomcat <-> MiniSpring bridge
│   └── nio/
│       └── ReactorServer.java        // optional Selector-based server
├── src/test/java/com/example/minitomcat/
│   ├── HttpRequestParserTest.java
│   ├── ServerThreadPoolTest.java
│   └── EndToEndSocketTest.java
```

Phase 1 — TCP Sockets Refresher

Everything a web server does sits on top of two classes: **ServerSocket** (a listening endpoint bound to a port) and **Socket** (one accepted TCP connection). **ServerSocket.accept()** **blocks** until a client connects, then returns a **Socket** you can read/write like any stream.

```
// The absolute minimum: bind, accept once, read nothing, write nothing, close.
try (ServerSocket serverSocket = new ServerSocket(8080)) {
    System.out.println("Listening on :8080");
    while (true) {
        Socket client = serverSocket.accept(); // blocks here
        client.close(); // we'll actually talk to it in the next phase
    }
}
```

Three facts drive the rest of this guide:

1. **accept()** hands you a new **Socket** per connection — the **ServerSocket** never talks to a client directly.
2. Reading/writing a **Socket**'s streams blocks the calling thread until bytes arrive or the buffer drains.

Phase 2 — TCP Sockets Refresher

```
public class EchoServer {
    public static void main(String[] args) throws IOException {
        try (ServerSocket serverSocket = new ServerSocket(8080)) {
            while (true) {
                try (Socket client = serverSocket.accept();
                    BufferedReader in = new BufferedReader(
                        new InputStreamReader(client.getInputStream()));
                    OutputStream out = client.getOutputStream()) {

                    String requestLine = in.readLine(); // e.g. "GET / HTTP/1.1"
                    System.out.println("Received: " + requestLine);

                    String body = "Hello from EchoServer";
                    String response =
                        "HTTP/1.1 200 OK\r\n" +
                        "Content-Type: text/plain\r\n" +
                        "Content-Length: " + body.length() + "\r\n" +
                        "Connection: close\r\n" +
                        "\r\n" +
                        body;
                    out.write(response.getBytes(StandardCharsets.US_ASCII));
                    out.flush();

                }
            }
        }
    }
}
```

Phase 3 — Parsing the Request Line and Headers

```
// http/HttpRequest.java
public class HttpRequest {
    private final String method;
    private final String queryString;
    private final String httpVersion;

    private final String path;
    private final Map<String, String> headers;
    private final byte[] body;

    public HttpRequest(String method, String path, String queryString,
        String httpVersion, Map<String, String> headers, byte[] body) {
        this.method = method;
        this.path = path;
        this.queryString = queryString;
        this.httpVersion = httpVersion;
        this.headers = headers;
        this.body = body;
    }

    public String getMethod() { return method; }
    public String getPath() { return path; }
    public String getQueryString() { return queryString; }
    public String getHeader(String name) { return headers.get(name.toLowerCase()); }
    public byte[] getBody() { return body; }
}
```

```
// http/HttpRequestParser.java
public class HttpRequestParser {
    public HttpRequest parse(InputStream rawIn) throws IOException {
        BufferedInputStream in = new BufferedInputStream(rawIn);

        String requestLine = readLine(in);
        if (requestLine == null || requestLine.isBlank()) {
            throw new MalformedRequestException("Empty request line");
        }
        String[] parts = requestLine.split(" ");
        if (parts.length != 3) {
            throw new MalformedRequestException("Bad request line: " + requestLine);
        }
        String method = parts[0];
        String fullPath = parts[1];
        String httpVersion = parts[2];

        String path = fullPath;
        String query = "";
        int qIndex = fullPath.indexOf('?');
        if (qIndex >= 0) {
            path = fullPath.substring(0, qIndex);
            query = fullPath.substring(qIndex + 1);
        }

        Map<String, String> headers = new LinkedHashMap<>();
        String line;
        while ((line = readLine(in)) != null && !line.isEmpty()) {
            int colon = line.indexOf(':');
            if (colon < 0) continue; // tolerate malformed header line rather than fail the request
            String name = line.substring(0, colon).trim().toLowerCase();
            String value = line.substring(colon + 1).trim();
            headers.put(name, value);
        }

        byte[] body = readBody(in, headers);
        return new HttpRequest(method, path, query, httpVersion, headers, body);
    }
}
```

```

}

/** Reads one CRLF-terminated line without over-reading into the next line — critical for keep-alive. */
private String readLine(InputStream in) throws IOException {
    ByteArrayOutputStream buffer = new ByteArrayOutputStream();
    int prev = -1, curr;
    while ((curr = in.read()) != -1) {
        if (prev == '\r' && curr == '\n') {
            byte[] bytes = buffer.toByteArray();
            return new String(bytes, 0, bytes.length - 1, StandardCharsets.US_ASCII);
        }
        buffer.write(curr);
        prev = curr;
    }
    return buffer.size() == 0 ? null : buffer.toString(StandardCharsets.US_ASCII);
}

private byte[] readBody(InputStream in, Map<String, String> headers) throws IOException {
    String transferEncoding = headers.get("transfer-encoding");
    if ("chunked".equalsIgnoreCase(transferEncoding)) {
        return readChunkedBody(in); // see §11
    }
    String contentLengthHeader = headers.get("content-length");
    if (contentLengthHeader == null) return new byte[0];

    int contentLength = Integer.parseInt(contentLengthHeader.trim());
    byte[] body = new byte[contentLength];
    int read = 0;
    while (read < contentLength) {
        int n = in.read(body, read, contentLength - read);
        if (n == -1) throw new MalformedURLException("Body shorter than Content-Length");
        read += n;
    }
    return body;
}

private byte[] readChunkedBody(InputStream in) throws IOException {
    ByteArrayOutputStream out = new ByteArrayOutputStream();
    while (true) {
        String sizeLine = readLine(in);
        int chunkSize = Integer.parseInt(sizeLine.trim(), 16); // chunk size is hex
        if (chunkSize == 0) {
            readLine(in); // consume the trailing empty line after the terminating 0-chunk
            break;
        }
        byte[] chunk = new byte[chunkSize];
        int read = 0;
        while (read < chunkSize) {
            int n = in.read(chunk, read, chunkSize - read);
            if (n == -1) throw new MalformedURLException("Truncated chunk body");
            read += n;
        }
        out.write(chunk);
        readLine(in); // consume the CRLF that follows every chunk's data
    }
    return out.toByteArray();
}
}

```

Why hand-roll readLine instead of BufferedReader.readLine()?

A **BufferedReader** wraps an **InputStreamReader** that pulls ahead and buffers decoded characters, which can silently consume bytes that belong to the body (especially a binary or chunked body) before you ever ask for them. Parsing HTTP requires precise control over exactly how many raw bytes have been consumed at every step — this is the same reason real servers implement their own buffered line-reading instead of trusting **BufferedReader** past the request line.

Handling the Request Body: Content-Length vs Chunked

Style	How the receiver knows where the body ends	When it's used
Content-Length: N	Read exactly N bytes.	Sender knows the full size upfront (e.g. a JSON payload already in memory).
Transfer-Encoding: chunked	Read size\r\n data\r\n blocks until a 0\r\n\r\n terminator.	Sender is streaming and doesn't know the total size yet (e.g. server-generated output).

A request must not specify both; if it does, most servers (and we do too, by construction above) prefer Transfer-Encoding since it is a hop-by-hop instruction about how to read the wire, while Content-Length is only a hint about total size.

Phase 4 — A Minimal Working HTTP Server

```
public class MiniTomcatV1 {
    public static void main(String[] args) throws IOException {
        HttpRequestParser parser = new HttpRequestParser();
        try (ServerSocket serverSocket = new ServerSocket(8080)) {
            while (true) {
                try (Socket client = serverSocket.accept()) {
                    handle(client, parser);
                } catch (IOException e) {
                    System.err.println("Connection error: " + e.getMessage());
                }
            }
        }
    }

    private static void handle(Socket client, HttpRequestParser parser) throws IOException {
        HttpRequest request = parser.parse(client.getInputStream());
        String body = "You requested " + request.getMethod() + " " + request.getPath();
        String response =
            "HTTP/1.1 200 OK\r\n" +
            "Content-Type: text/plain\r\n" +
            "Content-Length: " + body.getBytes(StandardCharsets.UTF_8).length + "\r\n" +
            "Connection: close\r\n\r\n" + body;
        client.getOutputStream().write(response.getBytes(StandardCharsets.UTF_8));
    }
}
```

Try it with `curl -v http://localhost:8080/hello`

Why Single-Threaded Fails Under Load

ServerSocket.accept() returns one connection at a time, and if the code handling that connection is slow (a database call, a slow client uploading a large body, network jitter), the accept loop can't call **accept()** again until it's done. Two clients become impossible to serve concurrently — the second waits behind the first even though the CPU is mostly idle.

Client A connects --> handle(A) takes 200ms (waiting on DB) --> accept() can't run --> Client B queues in the OS backlog

The backlog isn't infinite either — new `ServerSocket(port, backlog)` bounds how many pending connections the OS will hold before refusing new ones outright with a connection-reset error.

Phase 5 — Thread-Per-Connection Model

```
public class MiniTomcatThreaded {
    public static void main(String[] args) throws IOException {
        HttpRequestParser parser = new HttpRequestParser();
        try (ServerSocket serverSocket = new ServerSocket(8080)) {
            while (true) {
                Socket client = serverSocket.accept();
                Thread worker = new Thread(() -> handleSafely(client, parser));
                worker.start(); // fire-and-forget — this is the part we fix next
            }
        }

        private static void handleSafely(Socket client, HttpRequestParser parser) {
            try (client) {
                HttpRequest request = parser.parse(client.getInputStream());
                // ... build and write a response, same as §12 ...
            } catch (IOException e) {
                System.err.println("Worker failed: " + e.getMessage());
            }
        }
    }
}
```

This genuinely fixes concurrency — many clients can now be served in parallel, bounded only by CPU cores and blocking I/O wait time. It also introduces a new failure mode: **nothing bounds how many threads get created**.

The Cost of Unbounded Threads

Every new `Thread()` costs real, non-trivial resources:

Resource	Typical cost per thread	What happens at scale
Stack memory	~512 KB–1 MB (JVM default -Xss)	10,000 threads ≈ 5–10 GB of stack space reserved, before any request logic runs.
OS thread / kernel scheduling	1 native OS thread per Java platform thread	Context-switch overhead grows superlinearly as the scheduler juggles more runnable threads than cores.
File descriptors	1 per open socket	The OS has a hard ulimit -n; exceeding it makes <code>accept()</code> itself start throwing.
GC / safepoint pauses	Each live thread must reach a safepoint	More threads can lengthen stop-the-world pause coordination.

Phase 6 — Introducing a Thread Pool

```
// concurrent/ServerThreadPool.java
public class ServerThreadPool {
    private final ThreadPoolExecutor executor;

    public ServerThreadPool(int coreSize, int maxSize, int queueCapacity) {
        this.executor = new ThreadPoolExecutor(
            coreSize,
            maxSize,
            60L, TimeUnit.SECONDS,           // idle timeout for threads above coreSize
            new ArrayBlockingQueue<>(queueCapacity), // bounded queue — see §19
            new ThreadFactoryBuilder("mini-tomcat-worker"),
            new ThreadPoolExecutor.CallerRunsPolicy() // rejection policy — see §19
        );
    }

    public void submit(Runnable task) {
        executor.execute(task);
    }

    public void shutdown() {
        executor.shutdown();
    }
}
```

```
// A minimal named-thread factory so worker threads are identifiable in a thread dump.
class ThreadFactoryBuilder implements ThreadFactory {
    private final AtomicInteger counter = new AtomicInteger(1);
    private final String prefix;

    ThreadFactoryBuilder(String prefix) { this.prefix = prefix; }

    @Override
    public Thread newThread(Runnable r) {
        Thread t = new Thread(r, prefix + "-" + counter.getAndIncrement());
        t.setDaemon(false); // non-daemon: let in-flight requests finish before JVM exit
        return t;
    }
}
```

The accept loop no longer spawns threads itself — it just hands work to the pool:

```
try (ServerSocket serverSocket = new ServerSocket(8080)) {
    ServerThreadPool pool = new ServerThreadPool(20, 200, 1000);
    while (true) {
        Socket client = serverSocket.accept();
        pool.submit(() -> handleSafely(client, parser));
    }
}
```

Now the number of concurrently running handlers is capped, no matter how many clients connect — excess work waits in the pool's queue instead of spawning unbounded OS threads.

ThreadPoolExecutor Internals

ThreadPoolExecutor (what **Executors.newFixedThreadPool()** etc. return under the hood) makes this decision on every **execute(task)** call:

1. If fewer than `corePoolSize` threads exist -> start a new thread for this task, even if others are idle.
2. Else if the work queue has room -> enqueue the task; an existing thread will pick it up when free.
3. Else if fewer than `maximumPoolSize` threads exist -> start an extra ("non-core") thread for this task.
4. Else -> the pool is fully saturated: hand the task to the `RejectedExecutionHandler` (see §19).

Sizing the Pool: Core, Max, Queue, Keep-Alive

There is no universal formula, but a defensible starting point distinguishes CPU-bound from I/O-bound work:

```
threads = number_of_cores * (1 + wait_time / compute_time)
```

Workload	Rule of thumb	Reasoning
CPU-bound (hashing, serialization, template rendering)	<code>corePoolSize</code> $\approx$ CPU cores	More threads than cores just adds context-switch overhead with no extra throughput.
I/O-bound (DB calls, downstream HTTP, disk)	<code>corePoolSize</code> $\approx$ cores * (1 + wait/compute)	Threads spend most of their time blocked, not computing — more threads can be in flight concurrently.
<code>maximumPoolSize</code>	A hard ceiling, sized against available memory (maxSize * stackSize) and downstream capacity (e.g. DB pool size — no point running more request threads than your DB pool can serve).	Prevents an I/O-bound spike from still causing thread explosion.
<code>queueCapacity</code>	Bounded, sized to absorb short bursts (a few seconds of typical traffic) without either rejecting immediately or growing unboundedly.	An unbounded queue (<code>LinkedBlockingQueue</code> with no capacity) defeats <code>maximumPoolSize</code> entirely — the pool never grows past <code>corePoolSize</code> because step 3 above is never reached.
<code>keepAliveTime</code>	Seconds to a couple of minutes	How long a non-core thread sits idle before being reclaimed, trading thread-creation cost against idle memory.

Interview trap: "Just increase `maximumPoolSize`" is not free — it moves the bottleneck downstream (to the database, to a rate-limited API) and reintroduces the exact thread-explosion risk from §15 if the queue is also unbounded. Always size the pool and the queue and whatever the pool calls into, together.

Bounded Queues and Rejection Policies

When the queue is full and the pool is already at **maximumPoolSize**, `ThreadPoolExecutor` must decide what happens to the next submitted task — this is backpressure made concrete:

RejectedExecutionHandler	Behavior	When to use it for an HTTP server
<code>AbortPolicy</code> (default)	Throws RejectedExecutionException immediately.	You want the accept loop to know now that the server is saturated, e.g. to immediately return 503 Service Unavailable.
<code>CallerRunsPolicy</code>	Runs the task on the calling thread (here, the accept loop itself).	A simple, effective self-throttle: while the accept thread is busy running an overflow request, it can't call <code>accept()</code> again, naturally slowing new connection intake.
<code>DiscardPolicy</code>	Silently drops the task.	Almost never appropriate for a server — a dropped HTTP request looks like a hung connection to the client.
<code>DiscardOldestPolicy</code>	Drops the oldest queued task, then retries submission.	Rarely appropriate for requests (fairness violation — a request that's been waiting longest gets punished), more defensible for a metrics/logging queue.

Phase 7 — Wiring the Thread Pool Into the Server

```
public class MiniTomcat {
    private final ServerSocket serverSocket;
    private final ServerThreadPool pool;
    private final HttpRequestParser parser = new HttpRequestParser();
    private volatile boolean running = true;

    public MiniTomcat(int port, int coreThreads, int maxThreads, int queueCapacity) throws IOException {
        this.serverSocket = new ServerSocket(port);
        this.pool = new ServerThreadPool(coreThreads, maxThreads, queueCapacity);
    }

    public void start() {
        Thread acceptThread = new Thread(this::acceptLoop, "mini-tomcat-acceptor");
        acceptThread.start();
    }

    private void acceptLoop() {
        while (running) {
            try {
                Socket client = serverSocket.accept();
                pool.submit(new ConnectionWorker(client, parser));
            } catch (RejectedExecutionException saturated) {
                // Pool + queue both full and CallerRunsPolicy itself blocked too long — fail fast.
                writeServiceUnavailableAndClose(/* client */ null);
            } catch (IOException e) {
                if (running) System.err.println("Accept failed: " + e.getMessage());
            }
        }
    }

    public void stop() throws IOException {
        running = false;
        serverSocket.close(); // unblocks the pending accept()
        pool.shutdown();
    }
}
```

```
// server/ConnectionWorker.java
public class ConnectionWorker implements Runnable {
    private final Socket client;
    private final HttpRequestParser parser;

    public ConnectionWorker(Socket client, HttpRequestParser parser) {
        this.client = client;
        this.parser = parser;
    }

    @Override
    public void run() {
        try (client) {
            HttpRequest request = parser.parse(client.getInputStream());
            HttpResponse response = MiniServletContainer.getInstance().dispatch(request); // see §45
            client.getOutputStream().write(response.toBytes());
        } catch (IOException e) {
            System.err.println("Request failed: " + e.getMessage());
        }
    }
}
```

Graceful Shutdown

A production-grade **stop()** should not just call **executor.shutdown()** and exit — in-flight requests deserve a chance to finish, and hung ones need a deadline:

```
public void stop(Duration gracePeriod) throws IOException, InterruptedException {
    running = false;
    serverSocket.close();           // 1. stop accepting new connections
    pool.getExecutor().shutdown();  // 2. reject new submissions, let queued/running tasks finish
    boolean finished = pool.getExecutor()
        .awaitTermination(gracePeriod.toMillis(), TimeUnit.MILLISECONDS);
    if (!finished) {
        pool.getExecutor().shutdownNow(); // 3. interrupt anything still running past the deadline
    }
}
```

This three-step shutdown — stop intake, drain, then force — is the same pattern Kubernetes expects from a container that receives **SIGTERM**: stop accepting new work immediately, but give existing requests a bounded window to complete before the process is killed.

Why We Need Per-Request Context

Once requests run on pooled worker threads, a new problem appears: code deep inside a controller or repository (a logging statement, a security check, a **@Transactional**-style helper) often needs request-scoped data — the current user, a trace/correlation ID, the raw **HttpRequest** — without every method signature threading it through as an extra parameter.

```
// Without request context — every layer must pass traceId explicitly, forever
public User findUser(long id, String traceId) {
    log.info("{} looking up user {}", traceId, id);
    return repository.findById(id, traceId);
}
```

Since exactly one thread handles exactly one request at a time in the thread-per-request model (§14–§16), the thread itself is a natural place to stash that data — as long as two different requests running on the same reused pooled thread, one after another, never see each other's leftovers.

Introducing ThreadLocal

ThreadLocal<T> gives each thread its own independent copy of a variable — reads and writes from thread A never affect thread B's copy, even though they share the same **ThreadLocal** instance (the same static field).

```
ThreadLocal<String> currentUser = new ThreadLocal<>();

// Thread A:
currentUser.set("alice");
System.out.println(currentUser.get()); // "alice"

// Thread B (concurrently):
currentUser.set("bob");
System.out.println(currentUser.get()); // "bob" — completely independent of thread A's value
```

Internally, every **Thread** object carries a private **ThreadLocalMap** keyed by the **ThreadLocal** instance itself; **get()/set()** on a **ThreadLocal** are really "look up my map entry inside this thread's map." No locking is needed, because no two threads ever touch the same map.

Phase 8 — RequestContextHolder

```
// concurrent/RequestContextHolder.java
public final class RequestContextHolder {
    private static final ThreadLocal<RequestContext> CONTEXT = new ThreadLocal<>();

    private RequestContextHolder() {}

    public static void set(RequestContext context) {
        CONTEXT.set(context);
    }

    public static RequestContext get() {
        RequestContext context = CONTEXT.get();
        if (context == null) {
            throw new IllegalStateException("No request context bound to this thread");
        }
        return context;
    }

    public static void clear() {
        CONTEXT.remove(); // see §25 for why this call is not optional
    }
}

public class RequestContext {
    private final String traceId;
    private final HttpRequest request;
    private String currentUser; // set later, e.g. by an auth filter

    public RequestContext(String traceId, HttpRequest request) {
        this.traceId = traceId;
        this.request = request;
    }

    public String getTraceId() { return traceId; }
    public HttpRequest getRequest() { return request; }
    public void setCurrentUser(String user) { this.currentUser = user; }
    public String getCurrentUser() { return currentUser; }
}
```

ConnectionWorker.run() (§20) now binds and unbinds the context around the actual work:

```
@Override
public void run() {
    try (client) {
        HttpRequest request = parser.parse(client.getInputStream());
        RequestContextHolder.set(new RequestContext(UUID.randomUUID().toString(), request));
        HttpResponse response = MiniServletContainer.getInstance().dispatch(request);
        client.getOutputStream().write(response.toBytes());
    } catch (IOException e) {
        System.err.println("Request failed: " + e.getMessage());
    } finally {
        RequestContextHolder.clear(); // ALWAYS runs, even on exception — see §25–§26
    }
}
```

Now any code running on this thread during this request — no matter how deeply nested — can call `RequestContextHolder.get().getTraceId()` without a single extra method parameter. This is exactly how Spring's real `RequestContextHolder`

ThreadLocal and Thread Pools: The Leak Trap

A **ThreadLocal** value's lifetime is tied to the **thread**, not the **request**. In the thread-per-connection model of §14 that was harmless (the thread died with the connection). Once we introduced a pool in §16–§20, worker threads are **reused across many requests** — and that changes everything:

```
Request 1 runs on worker-thread-7 -> sets RequestContext(traceld="abc") -> forgets to clear()
Worker-thread-7 returns to the pool, waits idle
Request 2 is submitted, happens to run on worker-thread-7 -> calls RequestContextHolder.get()
-> gets Request 1's stale traceld "abc"! (a correctness bug: cross-request data leakage)
```

Worse, if the **RequestContext** (or anything it references — a large cached object, a security principal) is never cleared, it stays reachable for as long as the pooled thread lives, i.e. potentially the lifetime of the whole server process — a genuine **memory leak**, because the GC can never collect an object a live thread still references through its ThreadLocalMap.

This exact failure mode is a classic in application servers that reuse thread pools (Tomcat, Spring, Hibernate's ThreadLocal session) — it's precisely why frameworks are religious about clearing ThreadLocals at the end of every request.

Cleaning Up: try/finally and remove()

The fix is the one already shown in §24: **every** code path that sets a **ThreadLocal** for a request must clear it in a **finally** block, so it runs whether the request succeeded, threw an exception, or was cut off by a client disconnect.

```
RequestContextHolder.set(context);
try {
    handleRequest(context);
} finally {
    RequestContextHolder.clear(); // guaranteed to run
}
```

HTTP Keep-Alive Semantics

Every server so far has closed the socket after one response (Connection: close). Opening a fresh TCP connection per request — the three-way handshake, TLS negotiation if applicable, slow-start — is expensive relative to a tiny request/response pair. HTTP/1.1 makes persistent connections the default: unless a Connection: close header says otherwise, both sides keep the socket open and send another request/response pair over it.

```
Connection stays open:
Client -> GET /a -> Server responds -> (socket stays open)
Client -> GET /b -> Server responds -> (socket stays open)
Client -> GET /c -> Server responds -> ... until either side sends Connection: close, or an idle timeout fires
```

This changes our threading model's job: a worker thread handling a keep-alive connection isn't done after one response — it must loop, parsing another request from the same socket, until the client disconnects or times out.

Phase 9 — Implementing Keep-Alive

```
@Override
public void run() {
    try (client) {
        client.setSoTimeout(idleTimeoutMillis); // see §29
        while (!client.isClosed()) {
            HttpRequest request;
            try {
                request = parser.parse(client.getInputStream());
            } catch (SocketTimeoutException idle) {
                break; // no request arrived within the idle window — close politely
            } catch (EOFException clientClosed) {
                break; // client closed its write side — nothing more to read
            }

            RequestContextHolder.set(new RequestContext(UUID.randomUUID().toString(), request));
            boolean keepAlive;
            try {
                HttpResponse response = MiniServletContainer.getInstance().dispatch(request);
                keepAlive = shouldKeepAlive(request, response);
                response.setHeader("Connection", keepAlive ? "keep-alive" : "close");
                client.getOutputStream().write(response.toBytes());
                client.getOutputStream().flush();
            } finally {
                RequestContextHolder.clear();
            }
            if (!keepAlive) break;
        }
    } catch (IOException e) {
        System.err.println("Connection error: " + e.getMessage());
    }
}

private boolean shouldKeepAlive(HttpRequest request, HttpResponse response) {
    String clientHeader = request.getHeader("connection");
    if ("close".equalsIgnoreCase(clientHeader)) return false;
    if ("HTTP/1.0".equals(request.getHttpVersion())) {
        // HTTP/1.0 defaults to close unless the client explicitly opts in
        return "keep-alive".equalsIgnoreCase(clientHeader);
    }
    return true; // HTTP/1.1 defaults to keep-alive
}
```

Notice the worker thread is now occupied for the **entire lifetime of the connection**, not just one request — which is exactly why keep-alive and thread-pool sizing (§18) are coupled: a busy keep-alive client holds a pool thread hostage even while idle between requests, unless you either give it a short idle timeout (§29) or move to a model where one thread can serve many idle connections (§35–§38).

Idle Connection Timeouts and Reaping

`Socket.setSoTimeout(millis)` makes a blocking read throw **SocketTimeoutException** after that many milliseconds of no data — without it, a client that opens a connection and never sends anything (deliberately or by a network glitch) parks a worker thread forever, one of the simplest denial-of-service vectors against a thread-per-connection server.

```
client.setSoTimeout(30_000); // no data for 30s on an idle keep-alive connection -> give up gracefully
```

Two timeouts matter in practice, and real servers set them differently:

Timeout	Applies to	Typical value
Idle timeout (between requests on a keep-alive connection)	Time waiting for the next request line	Tens of seconds (Tomcat's connectionTimeout / keepAliveTimeout defaults are in this range).
Request timeout (mid-request)	Time to fully receive one request or send one response	Should generally be looser than the idle timeout, or handled separately, so a slow-but-legitimate upload isn't killed by the same clock as an idle keep-alive gap.

Two Meanings of "Connection Pool"

	Server-side "connection pool"	Client-side connection pool (e.g. JDBC)
What's being reused	An accepted Socket from a browser/client, kept alive across several HTTP requests (§27–§29).	An outbound Connection your server opens to something else (a database, another service), reused across several of your own outgoing calls.
Who initiates reuse	The remote client, via <code>Connection: keep-alive</code> .	Your own application code, via a pool object it calls borrow()/return() on.
What it saves	Repeated TCP handshakes with browsers.	Repeated TCP handshakes and expensive DB session/auth setup with the database.
Where it lives in MiniTomcat	<code>ConnectionWorker</code> + keep-alive loop (§28).	MiniConnectionPool (§32–§33), used by <code>MiniSpring @Repository</code> beans.

Phase 10 — A Generic Object Pool

Before building the JDBC-specific pool, it's worth building the reusable shape once, since MiniTomcat needs pooling in more than one place (buffers, parser scratch objects, DB connections):

```
// pool/ObjectPool.java
public interface ObjectPool<T> {
    T borrow() throws InterruptedException;
    void release(T item);
    void close();
}

// pool/BlockingObjectPool.java — a bounded pool backed by a BlockingQueue
public class BlockingObjectPool<T> implements ObjectPool<T> {
    private final BlockingQueue<T> available;
    private final Supplier<T> factory;

    public BlockingObjectPool(int size, Supplier<T> factory) {
        this.factory = factory;
        this.available = new ArrayBlockingQueue<>(size);
        for (int i = 0; i < size; i++) {
            available.offer(factory.get()); // pre-warm: create every instance up front
        }
    }

    @Override
    public T borrow() throws InterruptedException {
        return available.take(); // blocks here if the pool is fully checked out
    }

    @Override
    public void release(T item) {
        available.offer(item); // hand it back for the next borrower
    }

    @Override
    public void close() {
        available.clear();
    }
}
```

A **BufferPool** (pooled reusable **byte[]** buffers for reading request bodies) is then just new `BlockingObjectPool<>(64, () -> new byte[8192])` — avoiding a fresh 8 KB allocation (and the GC churn that goes with it) on every single request. The exact same shape, with a different factory, becomes the JDBC connection pool next.

Connection Pooling for the Database (JDBC)

Without pooling: every request pays connection setup cost

Request -> new Connection() [~20ms] -> query [~2ms] -> close() <- 90% of the time is setup!

With pooling: setup cost is paid once, amortized across many requests

Request -> pool.borrow() [~0ms, already open] -> query [~2ms] -> pool.release()

Phase 11 — A Minimal JDBC Connection Pool

```
// pool/jdbc/MiniConnectionPool.java
public class MiniConnectionPool implements ObjectPool<Connection> {
    private final BlockingQueue<Connection> available;
    private final Set<Connection> allConnections = ConcurrentHashMap.newKeySet();
    private final String url, user, password;

    public MiniConnectionPool(String url, String user, String password, int size) throws SQLException {
        this.url = url; this.user = user; this.password = password;
        this.available = new ArrayBlockingQueue<>(size);
        for (int i = 0; i < size; i++) {
            Connection connection = createRawConnection();
            allConnections.add(connection);
            available.offer(connection);
        }
    }

    private Connection createRawConnection() throws SQLException {
        return DriverManager.getConnection(url, user, password);
    }

    @Override
    public Connection borrow() throws InterruptedException {
        Connection connection = available.poll(); // non-blocking first, then wait
        if (connection == null) {
            connection = available.take(); // block until one is released — bounded, deliberate backpressure
        }
        return wrapAsPooled(connection);
    }

    /** Wraps close() so callers can use try-with-resources naturally without truly closing the socket. */
    private Connection wrapAsPooled(Connection real) {
        return (Connection) Proxy.newProxyInstance(
            getClass().getClassLoader(),
            new Class<?>[]{Connection.class},
            (proxy, method, args) -> {
                if ("close".equals(method.getName())) {
                    release(real); // return to the pool instead of actually closing the socket
                    return null;
                }
                return method.invoke(real, args);
            });
    }

    @Override
    public void release(Connection connection) { available.offer(connection); }

    @Override
    public void close() { allConnections.forEach(c -> try { c.close(); } catch (SQLException ignored) { }); }
}
```

Why the Proxy-based close() trick? It lets calling code keep writing idiomatic try (Connection c = pool.borrow()) { ... } — the try-with-resources block calls close() exactly as it would on a real connection, but that call is intercepted to mean "return to the pool" instead of "tear down the socket." This is the same illusion real pools like HikariCP maintain: application code never has to know it's holding a pooled wrapper.

Phase 12 — Swapping to Virtual Threads

```
public class ServerThreadPool {
    private final ExecutorService executor;

    public ServerThreadPool(Mode mode, int coreSize, int maxSize, int queueCapacity) {
        this.executor = switch (mode) {
            case PLATFORM_POOL -> new ThreadPoolExecutor(coreSize, maxSize, 60, TimeUnit.SECONDS,
                new ArrayBlockingQueue<>(queueCapacity));
            case VIRTUAL_THREADS -> Executors.newVirtualThreadPerTaskExecutor();
        };
    }

    public void submit(Runnable task) { executor.execute(task); }
    public void shutdown() { executor.shutdown(); }
}
```

The Servlet API in a Nutshell

```
public interface Servlet {
    void init(ServletConfig config) throws ServletException;
    void service(ServletRequest request, ServletResponse response) throws ServletException, IOException;
    void destroy();
    ServletConfig getServletConfig();
    String getServletInfo();
}
```

Phase 13 — Designing MiniServlet

```
// servlet/MiniServlet.java
public interface MiniServlet {
    void init(MiniServletConfig config) throws Exception;
    void service(MiniHttpServletRequest request, MiniHttpServletResponse response) throws Exception;
    void destroy();
}

// servlet/MiniServletConfig.java
public class MiniServletConfig {
    private final String servletName;
    private final MiniServletContext servletContext;
    private final Map<String, String> initParams;

    public MiniServletConfig(String servletName, MiniServletContext servletContext, Map<String, String>
initParams) {
        this.servletName = servletName;
        this.servletContext = servletContext;
        this.initParams = initParams;
    }
    public String getServletName() { return servletName; }
    public MiniServletContext getServletContext() { return servletContext; }
    public String getInitParameter(String name) { return initParams.get(name); }
}

// servlet/MiniServletContext.java — one instance shared by every servlet in the process
public class MiniServletContext {
    private final Map<String, Object> attributes = new ConcurrentHashMap<>();

    public void setAttribute(String name, Object value) { attributes.put(name, value); }
    public Object getAttribute(String name) { return attributes.get(name); }
}
```

MiniHttpServletRequest / MiniHttpServletResponse

```
// servlet/MiniHttpServletRequest.java
public class MiniHttpServletRequest {
    private final HttpRequest raw;
    private final Map<String, String> pathVariables = new HashMap<>(); // filled in by routing, §45/§50

    public MiniHttpServletRequest(HttpRequest raw) { this.raw = raw; }

    public String getMethod() { return raw.getMethod(); }
    public String getPathInfo() { return raw.getPath(); }
    public String getHeader(String name) { return raw.getHeader(name); }
    public String getParameter(String name) { return parseQueryParam(raw.getQueryString(), name); }
    public String getPathVariable(String name) { return pathVariables.get(name); }
    void setPathVariable(String name, String value) { pathVariables.put(name, value); }
    public byte[] getBody() { return raw.getBody(); }
    // parseQueryParam(...) omitted for brevity — splits on '&' and '='
}

// servlet/MiniHttpServletResponse.java
public class MiniHttpServletResponse {
    private int status = 200;
    private final Map<String, String> headers = new LinkedHashMap<>();
    private byte[] body = new byte[0];

    public void setStatus(int status) { this.status = status; }
    public void setHeader(String name, String value) { headers.put(name, value); }
    public void write(byte[] bytes) { this.body = bytes; }
    public void writeJson(String json) {
        setHeader("Content-Type", "application/json");
        write(json.getBytes(StandardCharsets.UTF_8));
    }

    HttpResponse toWireResponse() {
        HttpResponse response = new HttpResponse(status, body);
        headers.forEach(response::setHeader);
        response.setHeader("Content-Length", String.valueOf(body.length));
        return response;
    }
}
```

Phase 14 — MiniServletContainer (Path Routing)

```
public class MiniServletContainer {
    private static final MiniServletContainer INSTANCE = new MiniServletContainer();
    public static MiniServletContainer getInstance() { return INSTANCE; }
    private final Map<String, MiniServlet> pathToServlet = new LinkedHashMap<>(); // insertion order = priority
    private final MiniServletContext servletContext = new MiniServletContext();
    private final List<MiniFilter> filters = new ArrayList<>();

    public void register(String pathPattern, MiniServlet servlet, Map<String, String> initParams) {
        try {
            servlet.init(new MiniServletConfig(servlet.getClass().getSimpleName(), servletContext, initParams));
        } catch (Exception e) {
            throw new RuntimeException("Servlet init failed for " + pathPattern, e);
        }
        pathToServlet.put(pathPattern, servlet);
    }

    public void addFilter(MiniFilter filter) { filters.add(filter); }

    public MiniServletContext getServletContext() { return servletContext; }

    public HttpServletResponse dispatch(HttpRequest rawRequest) {
        MiniHttpServletRequest request = new MiniHttpServletRequest(rawRequest);
        MiniHttpServletResponse response = new MiniHttpServletResponse();

        MiniServlet servlet = resolve(rawRequest.getPath());
        if (servlet == null) {
            response.setStatus(404);
            response.write("Not Found".getBytes(StandardCharsets.UTF_8));
            return response.toWireResponse();
        }

        try {
            runFilterChainThenServlet(0, request, response, servlet);
        } catch (Exception e) {
            response.setStatus(500);
            response.write(("Internal Server Error: " + e.getMessage()).getBytes(StandardCharsets.UTF_8));
        }
        return response.toWireResponse();
    }

    private void runFilterChainThenServlet(int index, MiniHttpServletRequest req,
                                           MiniHttpServletResponse res, MiniServlet servlet) throws Exception {
        if (index < filters.size()) {
            filters.get(index).doFilter(req, res, () -> runFilterChainThenServlet(index + 1, req, res, servlet));
        } else {
            servlet.service(req, res);
        }
    }

    /** Longest-prefix match — the same basic strategy Tomcat uses for path-mapped servlets. */
    private MiniServlet resolve(String path) {
        return pathToServlet.entrySet().stream()
            .filter(e -> path.startsWith(e.getKey()))
            .max(Comparator.comparingInt(e -> e.getKey().length()))
            .map(Map.Entry::getValue)
            .orElse(null);
    }
}
```

Filters: Chain of Responsibility

```
// servlet/MiniFilter.java
@FunctionalInterface
public interface MiniFilterChain {
    void doFilter() throws Exception;
}

public interface MiniFilter {
    void doFilter(MiniHttpServletRequest request, MiniHttpServletResponse response, MiniFilterChain
chain) throws Exception;
}
```

```
// A logging filter — runs before AND after the rest of the chain
public class LoggingFilter implements MiniFilter {
    @Override
    public void doFilter(MiniHttpServletRequest req, MiniHttpServletResponse res, MiniFilterChain chain)
throws Exception {
        long start = System.nanoTime();
        try {
            chain.doFilter(); // runs the next filter, or finally the servlet
        } finally {
            long tookMs = (System.nanoTime() - start) / 1_000_000;
            System.out.printf("%s %s -> %d (%dms)%n", req.getMethod(), req.getPathInfo(),
res.getStatusForLogging(), tookMs);
        }
    }
}
```

Listeners: Lifecycle Hooks

Real Servlet containers also support `ServletContextListener` (container startup/shutdown) and `HttpSessionListener` (session created/destroyed) — hooks that run independent of any single request. A minimal equivalent:

```
public interface MiniServletContextListener {
    void contextInitialized(MiniServletContext context);
    void contextDestroyed(MiniServletContext context);
}
```

MiniTomcat calls every registered listener's **contextInitialized** once, right after building the **MiniServletContext** and before accepting the first connection — this is precisely the hook real Spring uses (**ContextLoaderListener**) to build the root **ApplicationContext** before any servlet's `init()` runs, guaranteeing beans exist by the time a request arrives. §50 reuses this same ordering for MiniSpring.

Phase 15 — MiniDispatcherServlet: Bridging the Two

```
public class MiniDispatcherServlet implements MiniServlet {
    public static final String APPLICATION_CONTEXT_ATTRIBUTE = "miniSpring.applicationContext";
    private ApplicationContext applicationContext; // MiniSpring's context interface
    private HandlerMapping handlerMapping;         // MiniSpring's route table

    @Override
    public void init(MiniServletConfig config) {
        // Retrieve the ApplicationContext a listener already built and published (§51) —
        // the servlet itself never calls new MiniApplicationContext(...) directly, mirroring
        // how real Spring's DispatcherServlet finds a context a ContextLoaderListener already built.
        this.applicationContext = (ApplicationContext)
            config.getServletContext().getAttribute(APPLICATION_CONTEXT_ATTRIBUTE);
        this.handlerMapping = new HandlerMapping();
        this.handlerMapping.registerControllers(applicationContext,
            ((MiniApplicationContext) applicationContext).getBeanFactory());
    }

    @Override
    public void service(MiniHttpServletRequest request, MiniHttpServletResponse response) {
        HandlerMethod handler = handlerMapping.find(request.getMethod(), request.getPathInfo());
        if (handler == null) {
            response.setStatus(404);
            response.writeJson("{\"error:No handler for " + request.getMethod() + " " + request.getPathInfo() + "\"}");
            return;
        }

        extractPathVariables(handler, request); // populate request.pathVariables from the matched pattern

        Object result = invokeHandler(handler, request);
        writeResult(response, result);
    }

    private Object invokeHandler(HandlerMethod handler, MiniHttpServletRequest request) {
        Method method = handler.getMethod();
        Object[] args = resolveArguments(method, request); // @PathVariable / @RequestParam,
        return method.invoke(handler.getControllerBean(), args);
    }

    private void writeResult(MiniHttpServletResponse response, Object result) {
        if (result instanceof ResponseEntity<?> entity) {
            response.setStatus(entity.getStatus());
            response.writeJson(JsonSerializer.toJson(entity.getBody()));
        } else {
            response.setStatus(200);
            response.writeJson(JsonSerializer.toJson(result));
        }
    }

    @Override
    public void destroy() { /* no-op: the ApplicationContext outlives this servlet */ }

    // extractPathVariables(...) / resolveArguments(...) reuse the exact logic already built in
    // MiniSpring's own FrontController (companion guide §31) — only the request/response TYPES differ,
    // since they now come from MiniHttpServletRequest/Response instead of com.sun.net.httpserver's
    // classes.
}
```